

화면 구성

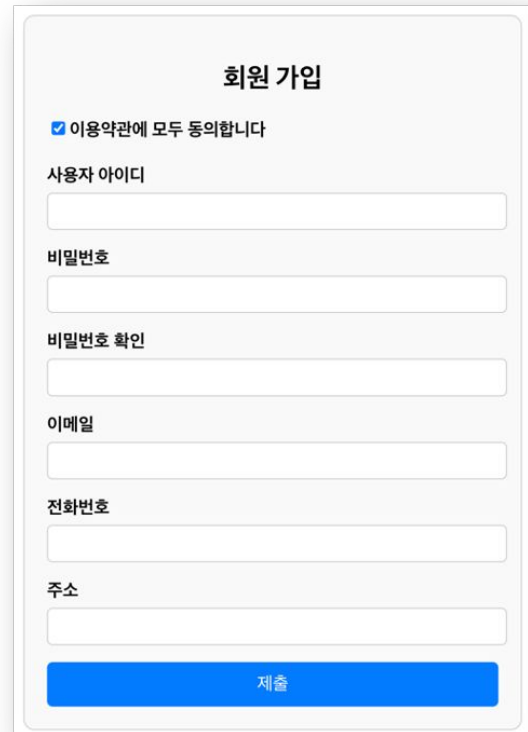
- 서버는 사용자와 소통해야 한다
 - 서버는 데이터를 처리하는 역할 외에도, 결과를 사용자에게 "보여주는" 역할이 필요함
 - 단순히 **JSON**이나 텍스트만 전달하면 사용자는 내용을 이해하기 어려움
 - 사용자는 눈으로 결과를 확인하고 행동해야 함
 - 눈으로 확인할 수 있는 화면(**UI**)은 사용자 경험의 핵심
 - 로그인 화면, 회원가입 폼, 검색 결과 등은 모두 시각적 피드백이 중요함
 - "로그인 성공"을 단순 텍스트로 보여주는 것보다 화면 전환이나 메시지 박스 등으로 제공하면 훨씬 이해하기 쉬움
 - 브라우저는 **HTML** 기반의 시각적 인터페이스에 최적화됨
 - 서버는 **HTML, CSS, JS** 등을 통해 사용자가 쉽게 사용할 수 있는 구조를 제공해야 함
 - **Node.js** 서버도 단순 **API** 응답뿐 아니라, 실제 웹페이지 형태로 응답하는 능력이 필요함
 - 특히 **Express**를 사용할 때는 다양한 응답 방법을 쉽게 설정 가능



First name:
Inje

Last name:
Lee

Submit



회원 가입

☒ 이용약관에 모두 동의합니다

사용자 아이디
[text input]

비밀번호
[text input]

비밀번호 확인
[text input]

이메일
[text input]

전화번호
[text input]

주소
[text input]

제출

□ Node.js에서 화면을 보여주는 세 가지 방식

○ 단순 문자열 응답 (res.send)

- HTML 태그를 포함한 문자열을 직접 작성해서 응답 (간단한 테스트 / 메시지)

○ 정적 HTML 파일 응답 (res.sendFile 또는 static 폴더)

- 미리 만든 HTML 파일을 그대로 전달

○ 템플릿 엔진을 이용한 응답 (ejs, pug 등)

- 서버 데이터와 HTML을 결합하여 동적으로 페이지 생성 (반복/조건문 등 다양한 표현 가능)

□ 단순 문자열

○ app-string.js

```
import express from "express";
```

```
const app = express();
```

```
app.get("/", (req, res) => {  
  res.send("<h1>Welcome!</h1><p>이 페이지는 단순 문자열로 만들어졌습니다.</p>");  
});  
app.listen(3000, () => {  
  console.log("서버 실행 중: http://localhost:3000");  
});
```

□ 정적 HTML

○ app-html.js

```
import express from "express";
```

```
import path from "path";
```

```
const app = express();
```

```
const __dirname = import.meta.dirname;
```

```
app.use(express.static(path.join(__dirname, "public")));
```

```
app.get("/", (req, res) => {  
  res.sendFile(path.join(__dirname, "public", "index.html"));  
});
```

```
app.listen(3000, () => {  
  console.log("서버 실행 중: http://localhost:3000");  
});
```

○ public/index.html

```
<h1>Welcome!</h1>
```

```
<p>이 페이지는 정적 HTML로 만들어졌습니다.</p>
```

□ EJS

- HTML 안에 JavaScript 코드를 삽입할 수 있는 템플릿 엔진
 - <% %> 문법으로 자바스크립트 코드 삽입
 - <%= %> 구문으로 변수 출력 가능

- 특징
 - HTML 문법을 그대로 사용할 수 있어 진입장벽이 낮음

- 설치: npm install ejs

- app-ejs.js

```
import express from "express";
```

```
import path from "path";
```

```
const __dirname = import.meta.dirname;
```

```
const app = express();
```

```
app.set("view engine", "ejs");
```

```
app.set("views", path.join(__dirname, "views"));
```

```
app.get("/", (req, res) => {  
  res.render("index", { name: "nodejs" });  
});
```

```
app.listen(3000, () => {  
  console.log("서버 실행 중: http://localhost:3000");  
});
```

- views/index.ejs

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <meta charset="UTF-8" />
```

```
    <title>EJS 예제 </title>
```

```
  </head>
```

```
  <body>
```

```
    <h1>환영합니다, <%= name %>님!</h1>
```

```
    <p>이 페이지는 EJS 템플릿으로 작성되었습니다.</p>
```

```
  </body>
```

```
</html>
```

연습문제

1. 사용자 환영 페이지 만들기 (`app-user.js`, `welcome.ejs`)

사용자 이름을 쿼리스트링으로 전달받아, EJS 템플릿을 이용해 환영 메시지를 출력하는 페이지를 작성합니다.

- 설명

- EJS 템플릿 엔진을 이용해 HTML과 서버 데이터를 결합하는 방식을 다룹니다.
- `res.render()` 로 템플릿 파일을 지정하고 그 템플릿 안에서 `<%= %>`로 서버에서 넘겨준 값을 출력합니다.

- 요구사항

- EJS 템플릿 엔진을 사용하여 동적 HTML 응답 구성
- `GET /welcome?name=사용자명` 형태로 요청 시, 사용자명을 포함한 환영 메시지를 출력
- 템플릿 파일(`welcome.ejs`)은 실행 파일과 같은 폴더에 위치

- 예상 동작



연습문제

1. 사용자 환영 페이지 만들기 (app-user.js, welcome.ejs)

[app-user.js]

```
import express from "express";
import path from "path";

const app = express();

// 코드 작성 1) EJS를 view engine으로 설정하고, views 폴더 경로 지정하기
// view engine → ejs
// views → app-user.js 와 같은 경로 (import.meta.dirname)

// 코드 작성 2) '/welcome' 경로 라우트 작성하기
// req.query.name으로 이름 꺼내기
// res.render("welcome", { name: 이름값 }) 으로 welcome.ejs 렌더링하기

app.listen(3000, () => {
  console.log("서버 실행 중: http://localhost:3000");
});
```

[welcome.ejs]

```
<body>
  <!-- 코드 작성 3) 전달받은 name 값을 이용해 환영 메시지 출력하기 -->
  <!-- <%= name %> 으로 서버에서 넘겨준 값을 화면에 출력 -->
</body>
```

EJS

□ EJS (Embedded JavaScript Templates)

- HTML 안에 JavaScript 코드를 삽입할 수 있도록 도와주는 템플릿 엔진
- 기존 HTML을 거의 그대로 유지하면서, 필요한 부분만 동적으로 바꿀 수 있어서 학습하기 쉬움

□ EJS에서 사용하는 기본 문법

- 변수 출력

`<%= 변수명 %>`

- HTML 안에서 JS 변수 값을 삽입할 때 사용

- 예시

`<h1>안녕하세요, <%= name %>님!</h1>`

- 자바스크립트 코드 실행 (출력 없음)

`<% JS 코드 %>`

- 조건문, 반복문 등 로직 실행용 (화면에 출력되지 않음)

- 예시

`<% if (isAdmin) { %>`

`<p>관리자 전용 페이지입니다.</p>`

`<% } %>`

- HTML escape 없이 출력 (raw 출력)

`<%- html코드 %>`

- `<%= %>`는 자동으로 HTML escape 처리됨 → `<` → `<`;

- `<%- %>`는 raw 출력 → HTML 태그가 실제로 반영됨

- 예시

`<%- "<h1>진짜 제목</h1>" %>`

□ 조건문

```
<% if (조건) { %>
  <p>참일 때 출력</p>
<% } else { %>
  <p>거짓일 때 출력</p>
<% } %>
```

- 완전히 일반적인 **JavaScript** 문법과 동일
- **if**, **else if**, **else** 모두 사용 가능

□ 반복문 (for, forEach)

```
<ul>
  <% for (let i = 0; i < items.length; i++) { %>
    <li><%= items[i] %></li>
  <% } %>
</ul>
```

```
<ul>
  <% items.forEach(item => { %>
    <li><%= item %></li>
  <% }) %>
</ul>
```

□ 주석 처리

```
<!-- 이걸 HTML에 출력되지 않는 EJS 주석 --%>
```

□ 조건 + 반복 + 변수 출력

```
<h1>사용자 목록</h1>
<ul>
  <% if (users.length === 0) { %>
    <li>사용자가 없습니다.</li>
  <% } else { %>
    <% users.forEach(user => { %>
      <li><%= user.name %> (<%= user.email %>)</li>
    <% }) %>
  <% } %>
</ul>
```

[실습]

EJS 템플릿으로 동적 화면 구성하기

1. app-template.js

```
import express from "express";
```

```
import path from "path";
```

```
const __dirname = import.meta.dirname;
```

```
const app = express();
```

```
// EJS 설정
```

```
app.set("view engine", "ejs");
```

```
app.set("views", path.join(__dirname, "views"));
```

```
// 테스트용 라우트: /test
```

```
app.get("/test", (req, res) => {
```

```
  res.render("template-test", {
```

```
    name: "nodejs",
```

```
    isAdmin: true,
```

```
    items: ["apple", "banana", "cherry"],
```

```
    rawHtml: "<strong style='color:red'>중요!</strong>"
```

```
  });
```

```
});
```

```
app.listen(3000, () => {
```

```
  console.log("✅ EJS 테스트 서버 실행 중: http://localhost:3000/test");
```

```
});
```


2. views/template-test.ejs

```
<!DOCTYPE html>

<html lang="ko">

<head>

  <meta charset="UTF-8" />

  <title>EJS 템플릿 테스트</title>

</head>

<body>

  <h1>안녕하세요, <%= name %>님</h1>


  <% if (isAdmin) { %>
    <p>🔑 관리자 계정으로 로그인 중입니다.</p>
  <% } else { %>
    <p>👤 일반 사용자입니다.</p>
  <% } %>


  <h2>장바구니 목록</h2>

  <ul>

    <% if (items.length === 0) { %>
      <li>항목이 없습니다.</li>
    <% } else { %>
      <% items.forEach(item => { %>
        <li><%= item %></li>
      <% }) %>
    <% } %>

  </ul>


  <h2>HTML 출력 테스트</h2>

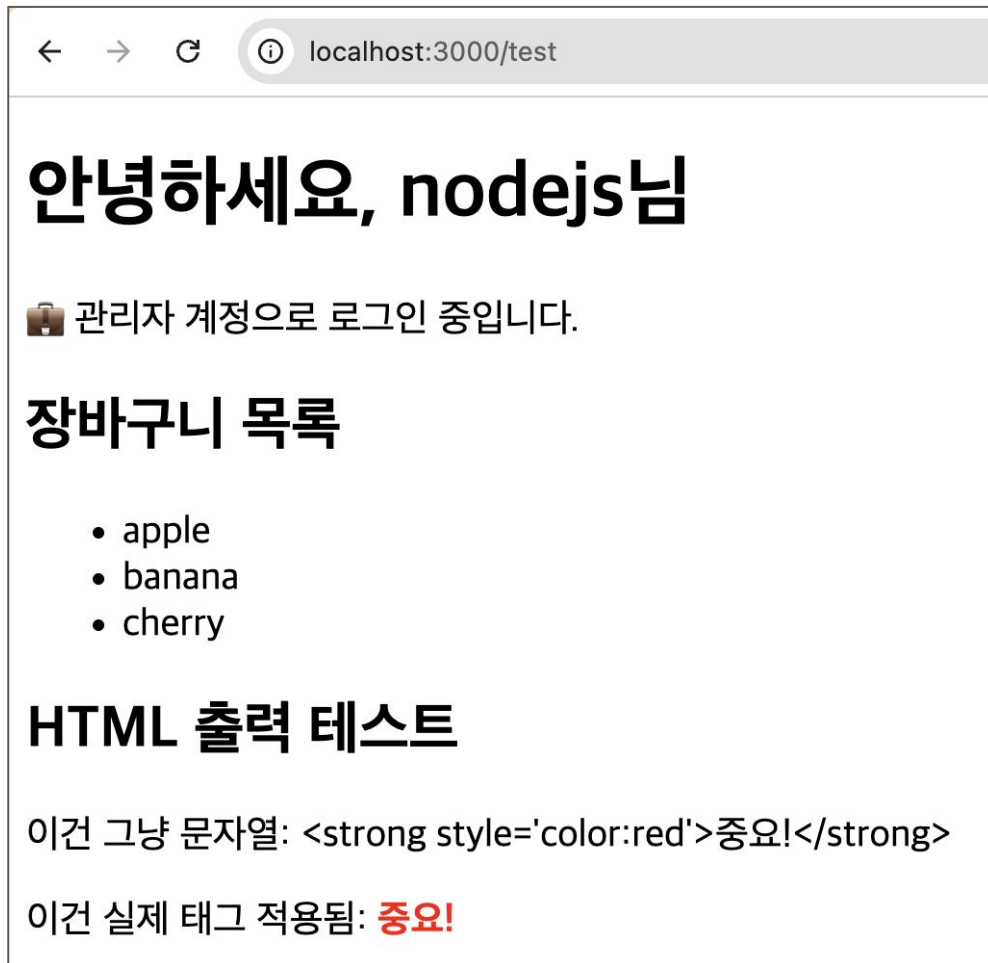
  <p>이건 그냥 문자열: <%= rawHtml %></p>

  <p>이건 실제 태그 적용됨: <%- rawHtml %></p>

</body>

</html>
```

3. 동작 확인



공통 레이아웃

□ EJS 공통 레이아웃

- 반복되는 **HTML** 구조를 재사용하여 유지보수를 쉽게 하기 위함
- 대부분의 페이지는 `<html>`, `<head>`, `<body>`, `<footer>` 구조가 동일
- 각 페이지마다 **본문**만 다르게 설정하고 나머지는 공통 코드로 유지

□ 공통 레이아웃을 구성하는 2가지 방법

- **express** 기본 기능
- **express-ejs-layouts** 모듈

구분	기본 기능 사용	express-ejs-layouts 모듈 사용
본문 연결 방식	layout.ejs 에서 본문 파일을 직접 지정 (<code><%- include(page) %></code>)	index.ejs 같은 개별 파일을 열면 자동으로 layout.ejs 로 감싸짐
본문 내용 출력	<code><%- include(page) %></code> 로 경로 전달	<code><%- body %></code> (고정 변수)
레이아웃 적용 방식	수동 적용	자동 적용
장점	원리 이해에 좋고 유연함	코드가 간결하고 유지보수 용이
단점	본문 경로를 일일이 관리해야 함	외부 모듈 의존, body 고정

- **index.ejs** 템플릿을 화면에 보여주는 경우

- 기본 기능

layout.ejs → **index.ejs**

```
<%- include('index') %>
```

- **express-ejs-layouts** 모듈

index.ejs → **layout.ejs**

□ 기본 기능

○ 구조

views/

|— layout1.ejs ← 공통 레이아웃

|— partials/

| |— header.ejs ← 공통 헤더

| |— footer.ejs ← 공통 푸터

|— index.ejs ← 본문 페이지

|— about.ejs ← 본문 페이지

|— contact.ejs ← 본문 페이지

app-layout1.js

○ 공통 레이아웃 파일: views/layout1.ejs

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <title><%= title %></title>
</head>
<body>
  <%- include('partials/header') %>

  <main>
    <%= include(page) %>
  </main>

  <%- include('partials/footer') %>
</body>
</html>
```



- 공통 헤더: `views/partials/header.ejs`

```
<header>
  <h1>My Website</h1>
  <nav>
    <a href="/">Home</a> |
    <a href="/about">About</a> |
    <a href="/contact">Contact</a>
  </nav>
  <hr>
</header>
```

My Website

[Home](#) | [About](#) | [Contact](#)

- 공통 푸터: `views/partials/footer.ejs`

```
<hr>
<footer>
  <p>&copy; 2025 nodejs 강의. All rights reserved.</p>
</footer>
```

© 2025 nodejs 강의. All rights reserved.

- 본문 페이지: `index.ejs`

```
<h2>홈페이지에 오신 것을 환영합니다!</h2>
<p>이 페이지는 EJS 공통 레이아웃을 사용하고 있습니다.</p>
```

홈페이지에 오신 것을 환영합니다!

이 페이지는 EJS 공통 레이아웃을 사용하고 있습니다.

- 실행 파일: `app-layout1.js`

```
import express from 'express';
import path from 'path';
```

```
const __dirname = import.meta.dirname;
const app = express();
```

```
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));
```

```
app.get('/', (req, res) => {
  res.render('layout1', { title: '홈 페이지1', page: 'index' });
});
```

```
app.get('/about', (req, res) => {
  res.render('layout1', { title: '홈 페이지2', page: 'about' });
});
```

```
app.get('/contact', (req, res) => {
  res.render('layout1', { title: '홈 페이지3', page: 'contact' });
});
```

```
app.listen(3000, () => {
  console.log('✅ 서버 실행 중: http://localhost:3000');
});
```

□ express-ejs-layouts 모듈

○ 구조

views/

├─ layout2.ejs ← 공통 레이아웃

├─ partials/

│ └─ header.ejs ← 공통 헤더

│ └─ footer.ejs ← 공통 푸터

├─ index.ejs ← 본문 페이지

├─ about.ejs ← 본문 페이지

└─ contact.ejs ← 본문 페이지

app-layout2.js

○ 공통 레이아웃 파일: views/layout2.ejs

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <title><%= title %></title>
</head>
<body>
  <%- include('partials/header') %>

  <main>
    <%= body %>
  </main>

  <%- include('partials/footer') %>
</body>
</html>
```



□ express-ejs-layouts 설정: npm install express-ejs-layouts

- 공통 헤더: `views/partials/header.ejs`

```
<header>
  <h1>My Website</h1>
  <nav>
    <a href="/">Home</a> |
    <a href="/about">About</a> |
    <a href="/contact">Contact</a>
  </nav>
  <hr>
</header>
```

My Website

[Home](#) | [About](#) | [Contact](#)

- 공통 푸터: `views/partials/footer.ejs`

```
<hr>
<footer>
  <p>&copy; 2025 nodejs 강의. All rights reserved.</p>
</footer>
```

© 2025 nodejs 강의. All rights reserved.

- 본문 페이지: `index.ejs`

```
<h2>홈페이지에 오신 것을 환영합니다!</h2>
<p>이 페이지는 EJS 공통 레이아웃을 사용하고 있습니다.</p>
```

- 실행 파일: `app-layout2.js`

```
import express from 'express';
import expressLayouts from 'express-ejs-layouts';
import path from 'path';

const __dirname = import.meta.dirname;
const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));
app.use(expressLayouts); // 레이아웃 미들웨어 기본값: layout.ejs
// app.set('layout', 'layout'); // 기본 레이아웃을 임의로 지정

app.get('/', (req, res) => {
  res.render('index', { title: '홈페이지 1', layout: 'layout2' });
});
app.get('/about', (req, res) => {
  res.render('about', { title: '홈페이지 2', layout: 'layout2' });
});
app.get('/contact', (req, res) => {
  res.render('contact', { title: '홈페이지 3', layout: 'layout2' });
});

app.listen(3000, () => {
  console.log('✅ 서버 실행 중: http://localhost:3000');
});
```

홈페이지에 오신 것을 환영합니다!

이 페이지는 EJS 공통 레이아웃을 사용하고 있습니다.

□ EJS 공통 레이아웃 구성하기

- 반복되는 **HTML** 구조를 재사용하여 유지보수를 쉽게 하기 위함
 - 대부분의 페이지에서 **<html>**, **<head>**, **<body>**, **<footer>** 구조는 동일
 - 각 페이지마다 달라지는 본문만 부분적으로 교체되도록 설정 가능

□ 구조 설계 예시

views/

```
├── layout.ejs    ← 공통 레이아웃
├── partials/
│   ├── header.ejs ← 헤더 파트
│   └── footer.ejs  ← 푸터 파트
├── index.ejs
├── about.ejs
└── contact.ejs
```

□ express-ejs-layouts 설정: `npm install express-ejs-layouts`

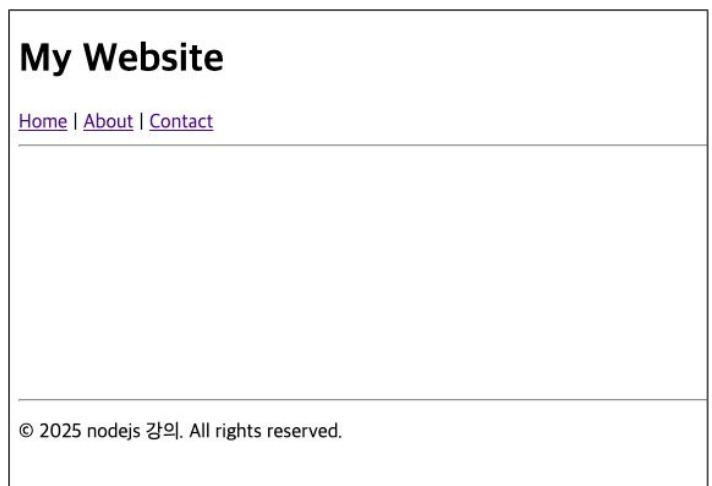
□ 공통 레이아웃 파일: `views/layout.ejs`

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <title><%= title %></title>
</head>
<body>
  <%- include('partials/header') %>

  <main>
    <%= body %> <!-- 개별 페이지 내용 삽입 -->
  </main>

  <%- include('partials/footer') %>

</body>
</html>
```



□ 부분 템플릿: `views/partials/header.ejs`

```
<header>
<h1>My Website</h1>
<nav>
  <a href="/">Home</a> |
  <a href="/about">About</a> |
  <a href="/contact">Contact</a>
</nav>
<hr>
</header>
```

My Website

[Home](#) | [About](#) | [Contact](#)

□ 부분 템플릿: `views/partials/footer.ejs`

```
<hr>
<footer>
  <p>&copy; 2025 nodejs 강의. All rights reserved.</p>
</footer>
```

© 2025 nodejs 강의. All rights reserved.

□ 각 페이지 본문: `views/index.ejs`

```
<h2>홈페이지에 오신 것을 환영합니다!</h2>
<p>이 페이지는 EJS 공통 레이아웃을 사용하고 있습니다.</p>
```

홈페이지에 오신 것을 환영합니다!

이 페이지는 EJS 공통 레이아웃을 사용하고 있습니다.

□ 각 페이지 본문: `views/about.ejs`

```
<h2>About 화면입니다!</h2>
<p>이 페이지는 EJS 공통 레이아웃을 사용하고 있습니다.</p>
```

About 화면입니다!

이 페이지는 EJS 공통 레이아웃을 사용하고 있습니다.

□ 각 페이지 본문: `views/contact.ejs`

```
<h2>Contact 화면입니다!</h2>
<p>이 페이지는 EJS 공통 레이아웃을 사용하고 있습니다.</p>
```

Contact 화면입니다!

이 페이지는 EJS 공통 레이아웃을 사용하고 있습니다.

□ 실행 파일: app-layout.js

```
import express from 'express';
```

```
import expressLayouts from 'express-ejs-layouts';
```

```
import path from 'path';
```

```
const __dirname = import.meta.dirname;
```

```
const app = express();
```

```
app.set('view engine', 'ejs');
```

```
app.set('views', path.join(__dirname, 'views'));
```

```
app.use(expressLayouts); // 레이아웃 미들웨어 기본값: layout.ejs
```

```
// app.set('layout', 'layout'); // 기본 레이아웃을 임의로 지정
```

```
app.get('/', (req, res) => {
```

```
  res.render('index', { title: '홈 페이지 1', layout: 'layout' });
```

```
});
```

```
app.get('/about', (req, res) => {
```

```
  res.render('about', { title: '홈 페이지 2', layout: 'layout' });
```

```
});
```

```
app.get('/contact', (req, res) => {
```

```
  res.render('contact', { title: '홈 페이지 3', layout: 'layout' });
```

```
});
```

```
app.listen(3000, () => {
```

```
  console.log('✅ 서버 실행 중: http://localhost:3000');
```

```
});
```

연습문제

1. 공통 레이아웃이 적용된 뉴스 페이지 만들기

- 설명

`<head>`, `<header>`, `<main>`, `<footer>` 로 구성되어 있는 `news.ejs` 를 공통 레이아웃으로 분리하기

- 요구사항

- `news.ejs` 를 아래의 구조로 분리
 - `views/layout.ejs`: 공통 레이아웃 (`<html>`, `<head>`, 본문이 들어갈 자리)
 - `views/partials/header.ejs`: `<header>` 영역만 분리
 - `views/partials/footer.ejs`: `<footer>` 영역만 분리
 - `views/news.ejs`: `<main>` 안의 내용만 남긴 얇은 파일로 축소

[원본] `views/news.ejs`

```
<head>
<meta charset="UTF-8">
<title><%= title %></title>
</head>

<body>
<header>
  <h1>My News</h1>
  <nav> <a href="/news">뉴스</a> </nav>
  <hr>
</header>
<main>
  <h2>최신 뉴스</h2>
  <ul>
    <% newsList.forEach(news=> { %>
      <li>
        <h3><%= news.title %></h3>
        <p><em><%= news.date %></em></p>
        <p><%= news.content %></p>
      </li>
    <% }) %>
  </ul>
</main>
<hr>
<footer>
  <p>&copy; 2025 Node.js 강의</p>
</footer>
</body>
```

My News

뉴스

최신 뉴스

- Node.js 22 출시

2025-06-01

성능이 더욱 향상된 Node.js 22가 정식 출시되었습니다.

- Express 5 정식 버전 공개

2025-05-20

10년만의 메이저 업그레이드로 많은 주목을 받고 있습니다.

© 2025 Node.js 강의

연습문제

1. 공통 레이아웃이 적용된 뉴스 페이지 만들기

[원본 news.js]

```
import express from "express";
import path from "path";

const app = express();

app.set("view engine", "ejs");
app.set("views", path.join(import.meta.dirname, "views"));

app.get("/news", (req, res) => {
  const newsItems = [
    { title: "Node.js 22 출시", date: "2025-06-01", content: "성능이 더욱 향상된 Node.js 22가 정식 출시되었습니다." },
    { title: "Express 5 정식 버전 공개", date: "2025-05-20", content: "10년만의 메이저 업그레이드로 많은 주목을 받고 있습니다." },
  ];

  res.render("news", {
    title: "뉴스 페이지",
    newsList: newsItems,
  });
});

app.listen(3000, () => {
  console.log("서버 실행 중: http://localhost:3000/news");
});
```

사용자 입력 처리

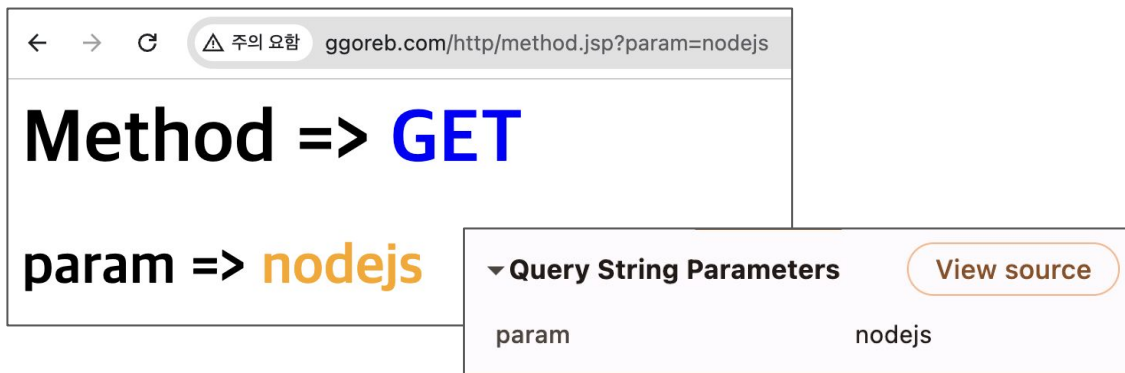
□ 사용자 입력은 어떻게 서버에 전달될까?

○ 사용자는 브라우저에서 입력(form)을 통해 서버와 소통

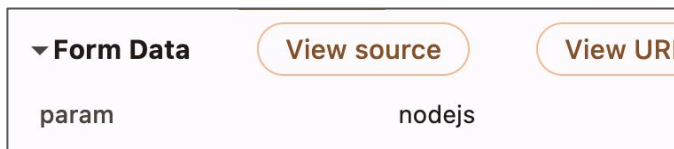
- 로그인, 회원가입, 댓글 작성 등
- HTML `<form>` 태그 사용
- JavaScript `fetch()` 함수 사용 등

○ 서버는 입력값을 요청(Request)의 쿼리(query) 또는 본문(body)으로 전달받음

- GET: 주소줄에 함께 전달



- POST: 본문(body)에 숨겨 전달



□ HTML Form 기본 구조

○ action / method / enctype 등 속성 사용

```
<form action="/submit" method="post">  
  <input type="text" name="username" />  
  <button>전송</button>  
</form>
```

- action: 데이터를 전송할 서버 주소(경로)
- method: 요청 방식 (GET / POST 등)
- name: 입력 데이터의 "키 이름" 역할

□ name 속성의 중요성

- name은 서버로 전송되는 데이터의 이름

`<input name="user">` → 서버로는 `user=`입력값 으로 전송됨

`<input name="email">` → `email=`입력값

- name이 없으면 서버는 해당 입력값을 인식하지 못함

□ GET 방식 – 주소(URL)에 데이터 포함 (req.query)

`<form action="/search" method="get">`

`<input name="q" />`

`<button>검색</button>`

`</form>`

→ 요청: `/search?q=apple`

→ 서버에서는: `req.query.q`로 꺼냄

□ POST 방식 – 요청 본문에 데이터 포함 (req.body)

`<form action="/login" method="post">`

`<input name="id" />`

`<input name="pw" />`

`<button>로그인</button>`

`</form>`

→ `Content-Type: application/x-www-form-urlencoded`

→ 서버에서는 `req.body.id`, `req.body.pw`로 꺼냄

※ POST 요청의 `body`를 읽기 위해서는 `express.urlencoded()` 미들웨어가 필요함

□ Express에서 입력값을 받기 위한 준비

- POST 요청의 본문(`body`)을 읽기 위해 미들웨어 필요

- `express.urlencoded()`

- `express.json()`

- 예시

`app.use(express.urlencoded({ extended: true }));`

`app.use(express.json());`

[실습]

사용자 입력 데이터 처리하기 (GET / POST)

1. app-form.js

```
import express from "express";
import path from "path";

const __dirname = import.meta.dirname;

const app = express();

app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views"));

app.use(express.urlencoded());

// 입력 폼 페이지
app.get("/form", (req, res) => {
  res.render("form");
});

// GET 방식 결과 처리
app.get("/form/result", (req, res) => {
  const { username } = req.query;
  res.render("result", { method: "GET", username });
});

// POST 방식 결과 처리
app.post("/form/result", (req, res) => {
  const { username } = req.body;
  res.render("result", { method: "POST", username });
});

app.listen(3000, () => {
  console.log("서버 실행 중: http://localhost:3000/form");
});
```

2. views/form.ejs

```
<head>
  <meta charset="UTF-8" />
  <title>입력 폼</title>
</head>
<body>
  <h1>사용자 입력 폼</h1>

  <h2>GET 방식</h2>
  <form action="/form/result" method="get">
    <input type="text" name="username" placeholder="이름 입력" />
    <button type="submit">전송 (GET)</button>
  </form>

  <h2>POST 방식</h2>
  <form action="/form/result" method="post">
    <input type="text" name="username" placeholder="이름 입력" />
    <button type="submit">전송 (POST)</button>
  </form>
</body>
```

사용자 입력 폼

GET 방식

전송 (GET)

POST 방식

전송 (POST)

3. views/result.ejs

```
<head>
  <meta charset="UTF-8" />
  <title>결과 페이지</title>
</head>
<body>
  <h1>입력 결과</h1>
  <p>요청 방식: <%= method %></p>

  <% if (username) { %>
    <p><strong><%= username %></strong>님, 환영합니다!</p>
  <% } else { %>
    <p>이름이 입력되지 않았습니다.</p>
  <% } %>
</body>
```

입력 결과

요청 방식: GET

이름이 입력되지 않았습니다.

[실습]

여러 항목 입력 및 라디오/체크박스 처리

1. app-form-advanced.js

```
import express from "express";
import path from "path";

const __dirname = import.meta.dirname;

const app = express();

app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views"));

app.use(express.urlencoded());

app.get("/form", (req, res) => {
  res.render("form-advanced");
});

app.post("/form/result", (req, res) => {
  const { username, gender, hobbies } = req.body;
  res.render("result-advanced", {
    username,
    gender,
    hobbies: Array.isArray(hobbies) ? hobbies : [hobbies].filter(value => Boolean(value))
  });
});

app.listen(3000, () => {
  console.log("서버 실행 중: http://localhost:3000/form");
});
```

2. views/form-advanced.ejs

```
<!DOCTYPE html>
```

```
<html lang="ko">
```

```
<head>
```

```
  <meta charset="UTF-8" />
```

```
  <title>확장된 입력 폼</title>
```

```
</head>
```

```
<body>
```

```
  <h1>회원 정보 입력</h1>
```

```
  <form action="/form/result" method="post">
```

```
    <p>
```

```
      이름: <input type="text" name="username" placeholder="이름을 입력하세요" />
```

```
    </p>
```

```
    <p>
```

```
      성별: <br />
```

```
      <input type="radio" name="gender" value="남성" /> 남성
```

```
      <input type="radio" name="gender" value="여성" /> 여성
```

```
    </p>
```

```
    <p>
```

```
      취미: <br />
```

```
      <input type="checkbox" name="hobbies" value="운동" /> 운동
```

```
      <input type="checkbox" name="hobbies" value="독서" /> 독서
```

```
      <input type="checkbox" name="hobbies" value="게임" /> 게임
```

```
    </p>
```

```
    <button type="submit">제출</button>
```

```
  </form>
```

```
</body>
```

```
</html>
```

회원 정보 입력

이름:

성별:

☐ 남성 ☐ 여성

취미:

☐ 운동 ☐ 독서 ☐ 게임

3. views/result-advanced.ejs

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <title>입력 결과</title>
</head>
<body>
  <h1>제출 결과</h1>

  <% if (username) { %>
    <p>이름: <%= username %></p>
  <% } %>

  <% if (gender) { %>
    <p>성별: <%= gender %></p>
  <% } %>

  <% if (hobbies && hobbies.length > 0) { %>
    <p>취미 목록:</p>
    <ul>
      <% hobbies.forEach(hobby => { %>
        <li><%= hobby %></li>
      <% }) %>
    </ul>
  <% } else { %>
    <p>취미를 선택하지 않았습니다.</p>
  <% } %>
</body>
</html>
```

제출 결과

이름: 이름

성별: 남성

취미 목록:

- 운동

연습문제

1. 점심 도시락 주문서 만들기 (`lunch-order.js`, `views/lunch-form.ejs`, `views/lunch-result.ejs`)

이름, 도시락 종류(라디오), 추가 반찬(체크박스)을 주문하면 EJS 템플릿으로 주문 내역을 출력하는 서버 작성

- 설명

- 라디오 버튼은 하나만 선택되므로 `req.body`에서 문자열 처리됩니다.
- 체크박스는 여러 개를 선택할 수 있어서 배열 또는 문자열로 들어올 수 있고 아무것도 선택하지 않은 경우 `undefined`로 들어올 수도 있습니다.

- 요구사항

- `GET /lunch/form` — 위 `lunch-form.ejs`를 보여줌
- `POST /lunch/result` — 제출된 값을 받아 결과 페이지 렌더링
- `username`(이름), `lunchbox`(도시락 종류), `sides`(추가 반찬)를 `req.body`에서 꺼내기
- 도시락 종류를 선택하지 않았으면 "도시락을 선택하지 않았습니다." 출력
- 추가 반찬을 하나도 선택하지 않았으면 "추가 반찬을 선택하지 않았습니다." 출력
- 반찬을 1개만 체크해도, 여러 개 체크해도 목록(`<ul>`)으로 정상 출력되어야 함

- 예상 동작

도시락 선택 X 반찬 0개 선택	도시락 선택 X 반찬 2개 선택
홍길동 님의 주문 결과 도시락 종류: 불고기 추가 반찬: • 계란말이 • 김치	홍길동 님의 주문 결과 도시락을 선택하지 않았습니다. 추가 반찬을 선택하지 않았습니다.

연습문제

1. 점심 도시락 주문서 만들기 (lunch-order.js, views/lunch-form.ejs, views/lunch-result.ejs)

```
[lunch-order.js]
import express from "express";
import path from "path";

const app = express();

app.set("view engine", "ejs");
app.set("views", path.join(import.meta.dirname, "views"));
app.use(express.urlencoded());

// 코드 작성 1) GET /lunch/form 라우트 작성하기
// lunch-form.ejs 렌더링

app.post("/lunch/result", (req, res) => {
  const { username, lunchbox, sides } = req.body;

  // 체크박스 처리
  // 어떤 형태로 들어오든 항상 배열 형태로 생성하는 코드
  const sidesList = Array.isArray(sides) ? sides : [sides].filter(Boolean);

  // 코드 작성 2) lunch-result.ejs 렌더링하기
  // username, lunchbox, sidesList를 전달

});

app.listen(3000, () => {
  console.log("서버 실행 중: http://localhost:3000/lunch/form");
});
```

연습문제

1. 점심 도시락 주문서 만들기 (lunch-order.js, views/lunch-form.ejs, views/lunch-result.ejs)

[views/lunch-result.ejs]

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <title>주문 결과</title>
</head>
<body>
  <h1><%= username %> 님의 주문 결과</h1>

  <!-- 코드 작성 4) 도시락 종류(lunchbox) 출력하기 -->
  <!-- lunchbox 값이 있으면 "도시락 종류: 값" / 없으면 "도시락을 선택하지 않았습니다." -->
  <% if (lunchbox) { %>
    <p>도시락 종류: <strong><%= lunchbox %></strong></p>
  <% } else { %>
    <p>도시락을 선택하지 않았습니다.</p>
  <% } %>

  <!-- 코드 작성 5) 추가 반찬(sides) 목록 출력하기 -->
  <!-- sides 배열 길이가 0보다 크면 forEach로 <li> 반복 출력 -->
  <!-- 없으면 "추가 반찬을 선택하지 않았습니다." -->
  <% if (sides.length > 0) { %>
    <p>추가 반찬:</p>
    <ul>
      <% sides.forEach(side => { %>
        <li><%= side %></li>
      <% }) %>
    </ul>
  <% } else { %>
    <p>추가 반찬을 선택하지 않았습니다.</p>
  <% } %>
</body>
</html>
```

유효성 검사

□ 유효성 검사란?

- 사용자가 입력한 값이 유효한지(문제없는 값인지) 확인하는 과정
 - 빈 칸인지?
 - 너무 짧거나 긴 값은 아닌지?
 - 선택하지 않은 항목은 없는지?
- 사용자가 실수할 수 있고 악의적으로 잘못된 값을 보낼 수도 있음
- 유효성 검사를 하지 않으면
 - 오류 발생 (예: DB 오류, 서버 다운)
 - 잘못된 데이터 저장
 - 보안 문제 발생 가능성 (ex: XSS, SQL Injection)

→ 그래서 모든 사용자 입력은 반드시 점검해야 됨

□ 유효성 검사 위치

위치	설명	예시
프론트엔드 (브라우저)	사용자 편의와 빠른 응답	alert 창, 빨간 테두리, required , JS 검사
백엔드 (서버)	최종 안전망. 무조건 필요	if (!username) { ... }, password.length > 20 등

□ 클라이언트 유효성 검사

- HTML에서 **required**, **maxlength**, **pattern** 사용
- JavaScript에서 **onsubmit**, **keyup** 등으로 검사

□ 서버 유효성 검사

- **req.body**에 담긴 값들을 조건으로 점검
- 필수 입력 확인, 글자 수 제한, 허용된 값인지 검사

[실습]

사용자가 입력한 로그인 정보 유효성 검사

1. app-login.js

```
import express from "express";
import path from "path";

const __dirname = import.meta.dirname;
const app = express();
app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views"));
app.use(express.urlencoded());

app.get("/login", (req, res) => {
  res.render("login", { error: null, values: {} });
});

app.post("/login", (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.render("login", {
      error: "아이디와 비밀번호를 모두 입력하세요.",
      values: req.body
    });
  }

  if (username.length > 20 || password.length > 20) {
    return res.render("login", {
      error: "입력값은 20자 이하만 가능합니다.",
      values: req.body
    });
  }

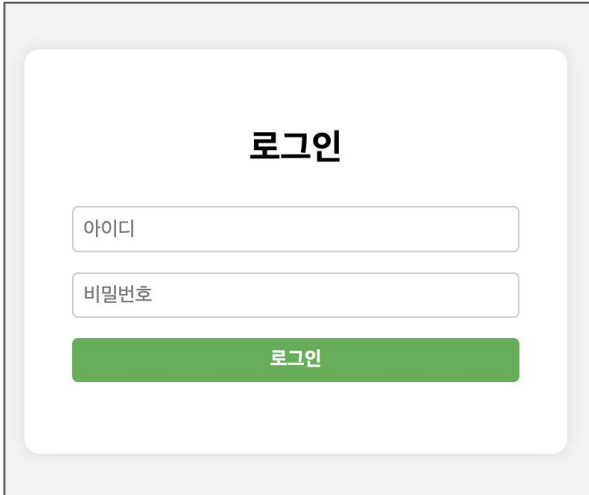
  if (username === "user" && password === "1234") {
    return res.render("welcome", { username });
  }

  return res.render("login", {
    error: "아이디 또는 비밀번호가 일치하지 않습니다.",
    values: req.body
  });
});

app.listen(3000, () => {
  console.log("✅ 서버 실행 중: http://localhost:3000/login");
});
```


2. views/login.ejs

```
<head>
<meta charset="UTF-8" />
<title>로그인</title>
<style>
  body {
    font-family: Arial;
    background: #f3f3f3;
    display: flex; height: 100vh;
    align-items: center; justify-content: center;
  }
  .login-box {
    background: white; padding: 2em;
    border-radius: 10px; width: 300px;
    box-shadow: 0 0 10px rgba(0,0,0,0.1);
  }
  h2 {
    text-align: center; margin-bottom: 1em;
  }
  input[type="text"],
  input[type="password"] {
    width: 100%; padding: 0.5em;
    margin-bottom: 1em;
    border: 1px solid #ccc;
    border-radius: 4px;
  }
  button {
    width: 100%; padding: 0.5em;
    background: #4CAF50;
    color: white; border: none;
    border-radius: 4px; font-weight: bold;
  }
  .error {
    color: red; margin-bottom: 1em; text-align: center;
  }
</style>
</head>
```



The image shows a visual representation of the login form defined in the code. It features a light gray background with a white rounded rectangle in the center. Inside the rectangle, the title '로그인' (Login) is centered at the top. Below the title are two input fields: the first is labeled '아이디' (ID) and the second is labeled '비밀번호' (Password). At the bottom of the form is a green button with the text '로그인' (Login) in white.

```

<body>
<div class="login-box">
  <h2>로그인</h2>
  <% if (error) { %>
    <div class="error"><%= error %></div>
  <% } %>
  <form action="/login" method="post" onsubmit="return validateForm()">
    <input type="text" name="username" placeholder="아이디"
      value="<%= values.username || " %>" />
    <input type="password" name="password" placeholder="비밀번호" />
    <button type="submit">로그인</button>
  </form>
</div>

<script>
function validateForm() {
  const username = document.querySelector('input[name="username"]').value.trim();
  const password = document.querySelector('input[name="password"]').value.trim();

  if (!username || !password) {
    alert("아이디와 비밀번호를 입력하세요.");
    return false;
  }

  if (username.length > 20 || password.length > 20) {
    alert("입력값은 20자 이하만 가능합니다.");
    return false;
  }

  return true;
}
</script>
</body>

```

로그인

아이디 또는 비밀번호가 일치하지 않습니다.

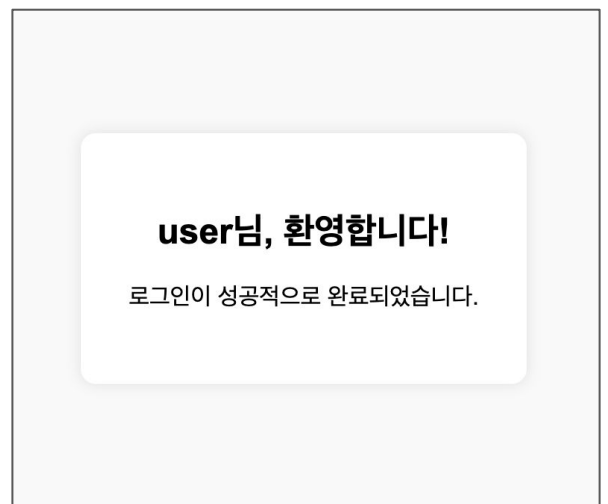
아이디

비밀번호

로그인

3. views/welcome.ejs

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <title>환영합니다</title>
  <style>
    body {
      font-family: Arial;
      background: #f9f9f9;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
    .welcome-box {
      background: #fff;
      padding: 2em;
      border-radius: 10px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
      text-align: center;
    }
  </style>
</head>
<body>
  <div class="welcome-box">
    <h2><%= username %>님, 환영합니다!</h2>
    <p>로그인이 성공적으로 완료되었습니다.</p>
  </div>
</body>
</html>
```



연습문제

1. 회원가입 화면 만들기 (signup.js, views/signup.ejs, views/signup-welcome.ejs)

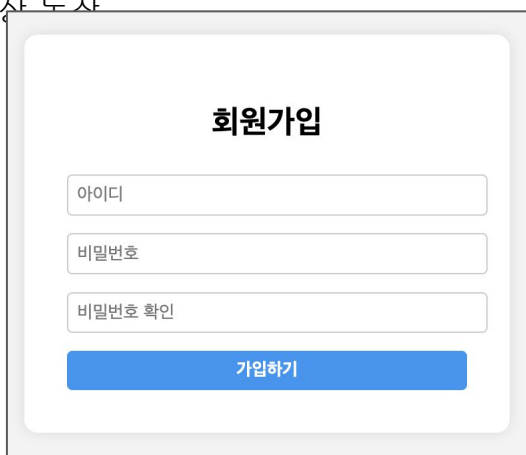
- 설명

회원가입 입력값 유효성 검사

- 요구사항

- GET /signup — 회원가입 폼 화면 보여주기
- POST /signup — 제출된 값(username, password, confirm)을 검사
- 프론트(JS)와 백엔드(Express) 양쪽에 아래 3가지 조건을 동일하게 적용
 - 입력값(username, password, confirm) 중 하나라도 비어 있으면 → "모든 항목을 입력하세요."
 - username 또는 password가 20글자를 초과하면 → "입력값은 20자 이하만 가능합니다."
 - password와 confirm이 서로 다르면 → "비밀번호가 일치하지 않습니다."
- 프론트 검사에 걸리면 alert()로 메시지를 띄우고 폼 제출을 막음 (return false)
- 백엔드 검사에 걸리면 에러 메시지와 함께 다시 signup.ejs를 렌더링 (입력했던 username 값은 유지)
- 모든 검사를 통과하면 signup-welcome.ejs로 이동

- 예상도자



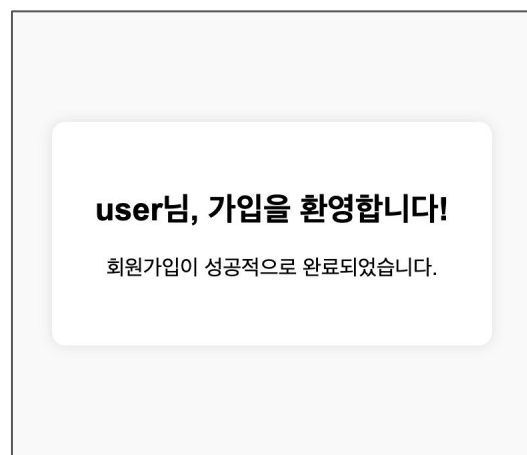
회원가입

아이디

비밀번호

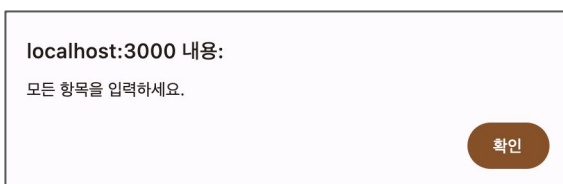
비밀번호 확인

가입하기



user님, 가입을 환영합니다!

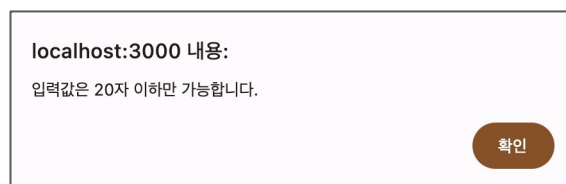
회원가입이 성공적으로 완료되었습니다.



localhost:3000 내용:

모든 항목을 입력하세요.

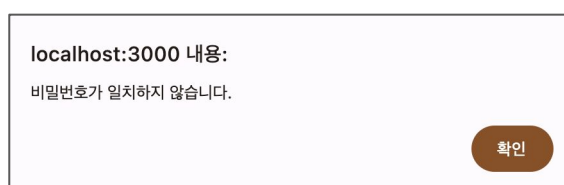
확인



localhost:3000 내용:

입력값은 20자 이하만 가능합니다.

확인



localhost:3000 내용:

비밀번호가 일치하지 않습니다.

확인

연습문제

1. 회원가입 화면 만들기 (signup.js, views/signup.ejs, views/signup-welcome.ejs)

```
[signup.js]
import express from "express";
import path from "path";

const app = express();
app.set("view engine", "ejs");
app.set("views", path.join(import.meta.dirname, "views"));
app.use(express.urlencoded());

// GET /signup - 폼 보여주기
app.get("/signup", (req, res) => {
  res.render("signup", { error: null, values: {} });
});

// POST /signup - 유효성 검사
app.post("/signup", (req, res) => {
  const { username, password, confirm } = req.body;

  // 코드 작성 1) 필수 입력 검사
  // username, password, confirm 중 하나라도 없으면
  // error: "모든 항목을 입력하세요." 로 signup.ejs 다시 렌더링 (values: req.body 유지)

  // 코드 작성 2) 글자 수 제한 검사
  // username 또는 password 길이가 20자를 초과하면
  // error: "입력값은 20자 이하만 가능합니다." 로 signup.ejs 다시 렌더링

  // 코드 작성 3) 비밀번호 확인 일치 검사
  // password와 confirm이 다르면
  // error: "비밀번호가 일치하지 않습니다." 로 signup.ejs 다시 렌더링

  // 성공 시
  res.render("welcome", { username });
});

app.listen(3000, () => {
  console.log("서버 실행 중: http://localhost:3000/signup");
});
```

연습문제

1. 회원가입 화면 만들기 (signup.js, views/signup.ejs, views/signup-welcome.ejs)

[views/signup.ejs]

```
<head>
<meta charset="UTF-8" />
<title>회원가입 </title>
<style>
  body {
    font-family: Arial; background: #f3f3f3; display: flex; height: 100vh;
    align-items: center; justify-content: center;
  }
  .signup-box {
    background: white; padding: 2em; border-radius: 10px;
    box-shadow: 0 0 10px rgba(0,0,0,0.1); width: 300px;
  }
  h2 { text-align: center; margin-bottom: 1em; }
  input {
    width: 100%; padding: 0.5em; margin-bottom: 1em; border: 1px solid #ccc; border-radius: 4px;
  }
  button {
    width: 100%; padding: 0.5em; background: #2196F3;
    color: white; border: none; border-radius: 4px; font-weight: bold;
  }
  .error { color: red; text-align: center; margin-bottom: 1em; }
</style>
</head>
<body>
<div class="signup-box">
  <h2>회원가입 </h2>
  <% if (error) { %>
    <div class="error"><%= error %></div>
  <% } %>
  <form action="/signup" method="post" onsubmit="return validateForm()">
    <input type="text" name="username" placeholder="아이디"
      value="<%= values.username || " %>" />
    <input type="password" name="password" placeholder="비밀번호" />
    <input type="password" name="confirm" placeholder="비밀번호 확인" />
    <button type="submit">가입하기 </button>
  </form>
</div>
```

연습문제

1. 회원가입 화면 만들기 (signup.js, views/signup.ejs, views/signup-welcome.ejs)

[views/signup.ejs]

```
<script>
function validateForm() {
  const username = document.querySelector('input[name="username"]').value.trim();
  const password = document.querySelector('input[name="password"]').value.trim();
  const confirm = document.querySelector('input[name="confirm"]').value.trim();

  // 코드 작성 4) 프론트 유효성 검사
  // 1) 셋 중 하나라도 비어있으면 alert("모든 항목을 입력하세요.") 후 false 반환
  // 2) username 또는 password가 20자 초과면 alert("입력값은 20자 이하만 가능합니다.") 후 false 반환
  // 3) password !== confirm 이면 alert("비밀번호가 일치하지 않습니다.") 후 false 반환

  return true;
}
</script>
</body>
```

localhost:3000 내용:

모든 항목을 입력하세요.

확인

localhost:3000 내용:

입력값은 20자 이하만 가능합니다.

확인

localhost:3000 내용:

비밀번호가 일치하지 않습니다.

확인

연습문제

1. 회원가입 화면 만들기 (signup.js, views/signup.ejs, views/signup-welcome.ejs)

[views/signup-welcome.ejs]

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <title>가입 완료</title>
  <style>
    body {
      font-family: Arial; background: #f9f9f9;
      display: flex; justify-content: center; align-items: center;
      height: 100vh;
    }
    .welcome-box {
      background: #fff; padding: 2em; border-radius: 10px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1); text-align: center;
    }
  </style>
</head>
<body>
  <div class="welcome-box">
    <h2><%= username %>님, 가입을 환영합니다!</h2>
    <p>회원가입이 성공적으로 완료되었습니다.</p>
  </div>
</body>
</html>
```

user님, 가입을 환영합니다!

회원가입이 성공적으로 완료되었습니다.

정적 리소스(static files) 사용하기

□ 정적 리소스

○ 정적 파일(static file)이란?

- 서버에서 별도의 처리 없이 그대로 전달되는 파일
- 예: 이미지(.jpg, .png), CSS, JavaScript, 폰트 파일 등

○ Express에서는 `express.static()` 미들웨어를 통해 정적 리소스 폴더를 지정할 수 있음

○ 기본 사용 방법

// 정적 파일 폴더 등록 (public 디렉토리)

```
app.use(express.static(path.join(__dirname, 'public')));
```

[실습]

css와 sample 이미지 사용

1. app-static.js

```
import express from 'express';
```

```
import path from 'path';
```

```
const __dirname = import.meta.dirname;
```

```
const app = express();
```

```
app.set('view engine', 'ejs');
```

```
app.set('views', path.join(__dirname, 'views'));
```

// 정적 파일 폴더 등록 (public 디렉토리)

```
app.use(express.static(path.join(__dirname, 'public')));
```

```
app.get('/static', (req, res) => {
```

```
  res.render('static');
```

```
});
```

```
app.listen(3000, () => {
```

```
  console.log('서버 실행 중: http://localhost:3000');
```

```
});
```

2. views/static.ejs

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <title>홈페이지</title>
  <link rel="stylesheet" href="/style.css" />
</head>
<body>
  <h1>Welcome to My Homepage!</h1>
  
</body>
</html>
```

3. public/style.css

```
body {
  background-color: #f0f8ff;
  font-family: Arial, sans-serif;
  text-align: center;
  padding: 2em;
}
h1 {
  color: #333;
}
img {
  width: 300px;
  border-radius: 8px;
}
```

4. public/sample.jpg



Welcome to My Homepage!